22 июня 2026 г.

Astro 7.0

Автор Мэтью Филлипс Эмануэле Стоппа Мэтт Кейн

Astro 7 уже здесь! В этом выпуске мы сделали упор на скорость. Компилятор `.astro` был переписан на языке Rust. Обработка Markdown и MDX теперь выполняется с помощью нового конвейера на языке Rust. Механизм рендеринга был заменен на более быстрый подход с использованием очередей. Вместе с Vite 8 и его новым сборщиком Rolldown сборка Astro 7 в наших тестах выполняется на 15–61 % быстрее. Самая быстрая сборка — это та, которая не выполняется вообще, поэтому в Astro 7 также стабилизировано кэширование маршрутов и добавлены экспериментальные провайдеры кэширования CDN для Netlify, Vercel и Cloudflare.

В Astro 7 также представлена расширенная маршрутизация, которая позволяет использовать `src/fetch.ts` точку входа с полным контролем над конвейером запросов Astro. Для разработки с использованием искусственного интеллекта Astro теперь может обнаруживать агентов-программистов, запускать сервер разработки в фоновом режиме и выводить структурированные журналы в формате JSON, когда агентам нужна машиночитаемая обратная связь.

Основные изменения в полной версии релиза:

- Vite 8
- Производительность
  - Компилятор Rust
  - Markdown и MDX в Rust
  - Рендеринг в очереди
- Расширенная маршрутизация
- Кэширование маршрутов
  - Провайдеры кэша CDN
- Улучшения в области искусственного интеллекта
  - Фоновый сервер разработки
  - Журналирование JSON
- Сообщество

Обновить сейчас

Чтобы обновить существующий проект до Astro 7, воспользуйтесь автоматизированным `@astrojs/upgrade` инструментом CLI:

```
# Recommended:
npx @astrojs/upgrade

# Manual:
npm install astro@latest
```

Для новых проектов просто используйте:

```
npm create astro@latest
```

Подробные инструкции по миграции см. в руководстве по обновлению.

Vite 8

Astro 7 обновляется до Vite 8, самого значительного релиза Vite за последние годы. Главное изменение: в Vite теперь используется Rolldown, бандлер на основе Rust, который заменяет esbuild и Rollup одним унифицированным бандлером. Rolldown в тестах в 10–30 раз быстрее, чем Rollup, при этом он поддерживает те же API плагинов Rollup и Vite.

Для пользователей Astro это означает более быструю сборку без изменений в конфигурации для большинства проектов. В Vite 8 есть уровень совместимости, который автоматически преобразует существующие `esbuild` и `rollupOptions` конфигурации в их аналоги для Rolldown. Если в вашем проекте используются пользовательские плагины Vite, большинство из них продолжат работать, поскольку Rolldown поддерживает тот же API плагинов, что и Rollup.

Performance

Astro 7 is the fastest version of Astro to date. As usage has grown, more teams have been pushing the boundaries of what type of site can be built with Astro. After we released Astro 6 and its big internal refactor, we set our sights on helping Astro scale to larger and more complex sites.

This is how Astro builds work:

1. Bundle the site's pages, content, and client components into JavaScript.
2. Run the bundled code like a little server, create requests for each prerendered page, and save the resulting HTML.

Astro 7 improves both steps, but focuses on the first: bundling the site. The biggest gains come from moving some of the slowest parts of the build into native code written in Rust. The generation step is also faster, thanks to a new rendering strategy that more effectively queues sections to be rendered.

In our testing, overall build times improved by 15–61%, with some sites building **more than twice as fast**. Sites where `.astro` compilation and Markdown processing make up a larger share of the build see the biggest gains, since those are exactly the parts that moved to Rust.

These benchmarks were run on a MacBook Pro Apple M4 Pro with 48 GB of memory:

Website	Before	After
https://docs.astro.build (~6,313 pages)	114.54s	73.53s
https://astro.build (~308 pages)	62.70s	24.24s
https://biomejs.dev (~6,488 pages)	176.39s	149.90s
https://developers.cloudflare.com (8,431 pages)	386.89s	261.94s
https://tauri.app (7,117 pages)	86.12s	55.33s
https://aspire.dev (13,275 pages)	385.84s	326.11s

Rust Compiler

We built a new compiler for the `.astro` component format, now written in Rust. The compiler is a full rewrite of our previous Go-based compiler that is mostly backwards compatible, except for:

No more HTML correction. The Go compiler silently rewrote your markup to be "valid HTML" by reordering elements, auto-closing tags, and moving nodes around in ways that often surprised users and caused hard-to-debug issues. The new compiler treats your markup as-is.

JSX-style strictness. Unclosed tags like `<div>Hello` and unterminated attributes like `<div class="Hello >` now produce errors instead of being silently corrected. These are real bugs in templates that the old compiler silently ignored in an attempt to behave like the browser.

JSX whitespace handling. Whitespace between elements is now collapsed following JSX conventions, matching the behavior of React and other JSX-based frameworks. For example, newlines between inline elements no longer produce a visible space:

```
<!-- Before: renders as "Hello World" (with a space) -->
<!-- After: renders as "HelloWorld" (no space) -->
<span>Hello</span>
<span>World</span>

<!-- To preserve the space, use an explicit expression: -->
<span>Hello</span>{' '}<span>World</span>
```

Moving to Rust allowed us to ship native binaries for supported platforms, with a WASM fallback for environments that need it. This pattern is now standard in the JavaScript tooling world, used by projects such as Rolldown and Lightning CSS. Under the hood, the new compiler is built on oxc for parsing and Lightning CSS for CSS scoping.

In isolation, the Rust compiler showed a roughly 6% improvement in build times on https://docs.astro.build. That's because `.astro` compilation is rarely the bottleneck; Markdown processing and bundling typically dominate build time. But every bit adds up, especially on large sites with thousands of pages, and the compiler's gains compound with the other performance improvements in this release.

Markdown & MDX in Rust

Astro 7 replaces the default Markdown and MDX pipeline with Sätteri, a Rust-powered processor created by Astro core team member  Erika. Switching the Astro docs and Cloudflare docs builds to Sätteri shaved over a minute off their build times, making Markdown-heavy sites the biggest winners in Astro 7.

Until now, Astro's Markdown pipeline ran on unified (remark, rehype, and a long tail of JavaScript dependencies). On large sites with thousands of pages, that pipeline was often the slowest phase of the build: each file parsed through JavaScript, run through plugin after plugin over the full AST, then serialized back to HTML. In Astro 6.4, we made the pipeline pluggable and shipped Sätteri as an opt-in alternative. Astro 7 makes it the default.

Under the hood, Sätteri uses pulldown-cmark for CommonMark parsing and Oxc for MDX expression parsing, both native Rust. It ships platform-specific binaries with a WASM fallback, the same approach used by the new `.astro` compiler. Speed isn't the only win, though. Sätteri also implements many Markdown features natively that previously required separate plugins:

Feature	unified	Sätteri
GFM (tables, footnotes, strikethrough, task lists)	remark-gfm plugin	Built-in, on by default
Smart punctuation (curly quotes, em dashes)	remark-smartypants plugin	Built-in

Feature	unified	Sätteri
Heading IDs	remark-heading-id or similar plugin	Built-in
Container directives	remark-directive plugin	Built-in
Math	remark-math plugin	Built-in
Frontmatter (YAML, TOML)	remark-frontmatter plugin	Built-in
Superscript / subscript	remark-supersub or similar plugin	Built-in
Wikilinks	remark-wiki-link plugin	Built-in

Non-default features are enabled through the `features` option:

astro.config.mjs

```
import { defineConfig } from 'astro/config';
import { satteri } from '@astrojs/markdown-satteri';

export default defineConfig({
  markdown: {
    processor: satteri({
      features: {
        directive: true,
        math: true,
        headingAttributes: true,
      },
    }),
  },
});
```

Sätteri has its own plugin API too. Plugins declare which node types they care about and skip the rest, instead of walking the entire tree on every pass. This makes adding plugins much cheaper than in unified, where every plugin traverses every node.

If you depend on remark or rehype plugins, the unified-based pipeline is still available via `@astrojs/markdown-remark`:

astro.config.mjs

```
import { defineConfig } from 'astro/config';
import { unified } from '@astrojs/markdown-remark';
import remarkToc from 'remark-toc';

export default defineConfig({
  markdown: {
    processor: unified({
      remarkPlugins: [remarkToc],
    }),
  },
});
```

Learn more about Sätteri's features and plugin API at satteri.bruits.org.

Queued Rendering

Queued rendering was introduced in Astro 6.0 as an experimental option and is now stable and the default rendering engine. It's ~2.4× faster[1] in speed!

astro.config.mjs

```
import { defineConfig } from "astro/config";

export default defineConfig({
  experimental: {
-   queuedRendering: {
-     enabled: true,
-     pooling: true,
-     contentCache: 1000
-   }
  }
})
```

Previously, Astro rendered pages using a recursive approach where children were rendered using the same `render*` function, like in the following pseudocode:

render.ts

```
export function renderComponentToString(node: unknown): string {
  let destination = "";
  destination += `<${node.name}>`; // opening tag
```

```
  // render attributes

  for (const child of node.children) {
    // Here's where we recurse the children by calling renderComponentToStri
    destination += renderComponentToString(child);
  }

  destination += `</${node.name}>`; // closing tag

  return destination;
}
```

The new engine uses a queue (or stack) and a single loop. The queue is populated with child nodes in the correct order, and the loop keeps rendering nodes until the queue is drained. The following pseudocode is an oversimplified version of the real deal, but it should provide a clear picture:

render.ts

```
export function renderComponentToString(root: unknown): string {
  let destination = "";
  destination += `<${root.name}>`; // opening tag

  // this is our queue, populated and flushed as we render
  let stack = [root];

  while (stack.length > 0) {
    const node = stack.pop();

    if (Array.isArray(node)) {
      // The nodes at the very end must be rendered to destination first
      for (let i = node.length - 1; i >= 0; i--) stack.push(node[i]);
      continue;
    }

    const nodeType = typeof node;
    if (nodeType === 'string') {
      destination += escapeHTML(node as string);
    }
  }
  destination += `</${root.name}>`; // opening tag
  return destination;
```

```
}
```

The first implementation of the strategy worked on a two-pass phase: create an ordered list of components (nodes), and loop the list and render the components.

The new implementation doesn't create a full list anymore, instead the list is rendered (flushed) while it's looped. This final approach is faster than the first iteration, and needs less memory compared to the recursive approach.

Advanced Routing

Astro started as a static site generator with file-based routing. Over time, features like middleware, redirects, rewrites, Actions, sessions, and i18n gave Astro apps more server-side power, but they also made the request lifecycle harder to control. If you needed auth to run before Actions, logging to wrap only page rendering, or a non-Astro API to handle some requests first, you had to work around the pipeline instead of composing it directly.

In Astro 7, you can now take full control over Astro's request pipeline by adding a `src/fetch.ts` file to your project. This file exports the standard `fetch` handler pattern popularized by Cloudflare Workers, Deno, and Bun.

src/fetch.ts

```ts
import { astro, FetchState } from 'astro/fetch';

export default {
  fetch(request: Request) {
    const state = new FetchState(request);

    // Forward API requests to a backend service
    if (state.url.pathname.startsWith('/api')) {
      const url = new URL(state.url.pathname + state.url.search, 'https://ba
      return fetch(new Request(url, request));
    }

    // Fallback to Astro pages/endpoints
    return astro(state);
  }
}
```

The API is also compatible with Hono, allowing you to bring Hono middleware into your Astro application:

src/fetch.ts

```ts
import { astro } from 'astro/hono';
import { Hono } from 'hono';
import { basicAuth } from 'hono/basic-auth';

const app = new Hono();
app.use(basicAuth({ username: 'admin', password: 'secret' }));
app.use(astro());

export default app;
```

For advanced usage, you can compose individual Astro features as separate middleware, giving you full control over the request pipeline. If you've ever used Astro middleware and been frustrated that your auth check ran after Astro Actions, or that you couldn't log response timing without wrapping everything yourself, you can now put your code exactly where it needs to be:

src/fetch.ts

```ts
import { Hono } from 'hono';
import { actions, middleware, pages, i18n } from 'astro/hono';
import { auth } from './middleware/auth';
import { timing } from './middleware/timing';

const app = new Hono();

app.use(i18n());
app.use(auth());       // Auth runs before actions, no unauthenticated calls
app.use(actions());
app.use(middleware());
app.use(timing());     // Timing wraps only page rendering
app.use(pages());

export default app;
```

If you don't add a `src/fetch.ts` file, Astro behaves exactly as it does today.

Route Caching

Caching on-demand rendered responses is harder than it should be. Every host does it differently, and there has never been a standard way to control it from your application code. Astro 7 introduces route caching to solve this. First released experimentally in Astro 6, the feature is now stable and adds a single platform-agnostic API for caching: set directives in your routes, and Astro handles the rest wherever you deploy.

You configure a cache provider once, then use `Astro.cache` in your pages (or `context.cache` in API routes and middleware) to control caching per response, based on standard HTTP caching semantics. Astro ships with a built-in `memoryCache()` provider to get you started:

astro.config.mjs

```
import { defineConfig, memoryCache } from 'astro/config';

export default defineConfig({
  cache: {
    provider: memoryCache(),
  },
});
```

src/pages/products/[id].astro

```
---
Astro.cache.set({
  maxAge: 120, // Cache for 2 minutes
  swr: 60, // Serve stale for 1 minute while revalidating
  tags: ['products'], // Tag for targeted invalidation
});
---
```

You can also define caching rules for groups of routes declaratively in your config with `routeRules`, keeping caching out of your route code entirely:

astro.config.mjs

```
export default defineConfig({
```

```
  cache: { provider: memoryCache() },
  routeRules: {
    '/blog/[...path]': { maxAge: 300, swr: 60 },
  },
});
```

Where route caching really shines is its integration with live content collections. A live loader can attach a cache hint to the data it returns, with tags for invalidation and a last-modified time for freshness. Pass that entry straight to `Astro.cache.set()` and Astro reads the hint for you, no manual headers required:

src/pages/products/[id].astro

```
---
import { getLiveEntry } from 'astro:content';
const { entry } = await getLiveEntry('products', Astro.params.id);

// Astro reads the loader's cache hint from the entry:
Astro.cache.set(entry);
---
```

Cached responses can be purged on demand with `cache.invalidate()`, by tag or by path. For example, you can expose a webhook endpoint for your CMS to call whenever content changes. This maps to each provider's invalidation API, clearing every affected response without a rebuild:

src/pages/api/revalidate.ts

```
import type { APIRoute } from 'astro';

export const POST: APIRoute = async ({ request, cache }) => {
  // A real implementation would validate the request and check a secret tok

  const { slug } = await request.json();

  // Invalidate every response tagged 'products'...
  await cache.invalidate({ tags: ["products"] });

  // ...invalidate every page that used a particular entry...
  await cache.invalidate({ tags: [`products:${slug}`] });

  // ...or purge a single path directly.
```

```
  await cache.invalidate({ path: `/products/${slug}` });

  return new Response('Revalidated');
};
```

If you tried route caching while it was experimental, the only change is to move `cache` and `routeRules` out of the `experimental` block to the top level of your config. The API is otherwise unchanged. See the route caching guide for the full reference.

CDN Cache Providers

When route caching shipped in Astro 6, it came with a single in-memory provider to use with the Node adapter. Astro 7 adds experimental CDN providers for Netlify, Vercel, and Cloudflare (in private beta).

Rather than storing responses in memory, these providers push your caching directives down to the host's edge network, for even faster responses. Cache hits are then served straight from the CDN, without invoking your server function at all.

In a future release, these providers will be enabled automatically, but during the experimental phase you should add them manually. Import the provider for your adapter and set it as your cache provider:

astro.config.mjs

```
import { defineConfig } from 'astro/config';
import netlify from '@astrojs/netlify';
import { cacheNetlify } from '@astrojs/netlify/cache';

export default defineConfig({
  adapter: netlify(),
  cache: {
    provider: cacheNetlify(),
  },
});
```

Each adapter exports a provider from its `/cache` entrypoint:

Adapter	Import	Provider
Netlify	`@astrojs/netlify/cache`	`cacheNetlify()`
Vercel	`@astrojs/vercel/cache`	`cacheVercel()`
Cloudflare ⚠️	`@astrojs/cloudflare/cache`	`cacheCloudflare()`

The same `Astro.cache`, `routeRules`, and `cache.invalidate()` APIs work with every provider. Each one translates your directives into the platform's native cache-control headers and tag- or path-based purges.

PRIVATE BETA

The Cloudflare provider requires the Cloudflare Workers Cache feature, which is currently in private beta. Without beta access, it does not work and should not be used.

AI Enhancements

AI coding agents are now part of many developers' workflows, and they need different things from a dev server than humans do. Astro 7 is our first step toward making Astro a better platform for agent-driven development.

Background Dev Server

AI agents struggle with long-running processes. They shell out, wait for exit, and read output, but a dev server never exits. Agents hang, start duplicate servers, lose track of running instances, or leave zombie processes behind. We see this as part of Astro's automated issue triage as well, with agents sometimes spending more time fumbling with dev servers than testing code.

Astro 7 adds `astro dev --background`, which starts the dev server as a managed background process. **Astro can also detect when it's running inside an AI agent** and enable background mode automatically, so no flags are needed in agent workflows. If no agent is detected, `astro dev` behaves exactly as before.

```
$ astro dev --background
Dev server running at http://localhost:4321 (pid 12345)
  Stop:   astro dev stop
  Status: astro dev status
  Logs:   astro dev logs
```

The command blocks until the server is ready to accept requests, reports the URL and process ID, then detaches. No polling, no sleeping, no parsing terminal output for "Local:".

A **lockfile** prevents duplicate instances. If an agent tries to start a second server, it gets back the existing instance's details instead of spawning a conflicting process:

```
$ astro dev --background
Dev server already running at http://localhost:4321 (pid 12345)
```

You can check status and stop the server from separate shell sessions:

```
$ astro dev status
Dev server running at http://localhost:4321 (pid 12345, uptime 123s, backgro

$ astro dev stop
Stopped dev server (pid 12345).
```

You can also read the background server's logs with `astro dev logs`.

Every command is idempotent and forgiving. Stopping when not running succeeds silently, and starting when already running returns the existing instance. Agents often lose track of process state, and the CLI doesn't punish them for it.

All running dev servers also expose a `/_astro/status` health endpoint that agents can query to confirm the server is alive and ready to accept requests.

JSON Logging

Astro's logger is now fully configurable. For agents, JSON logging is enabled automatically when agent detection turns on background mode. For everyone else, it's

available via the CLI or configuration:

```
astro dev --json
```

astro.config.mjs

```
import { defineConfig, logHandlers } from "astro/config";

export default defineConfig({
  logger: logHandlers.json()
})
```

JSON logging was the most upvoted feature request on our roadmap, and not just because of AI. Teams deploying Astro SSR to production need structured logs for integration with log aggregation services like Kibana, CloudWatch, and Grafana/Loki. Astro's previous logging was hardcoded for human readability: colors, box-drawing characters, multi-line error formatting. None of that is parseable by machines.

The new logger API also supports custom log handlers and a `compose()` API for combining multiple loggers. For example, you can keep human-readable output in the console while also writing JSON logs for tools that need structured output:

astro.config.mjs

```
import { defineConfig, logHandlers } from "astro/config";

export default defineConfig({
  logger: logHandlers.compose(
    logHandlers.console(),
    logHandlers.json()
  )
})
```

Learn more about Astro's support for AI tooling in the AI guide.

Community

The Astro core team is:

Alexander Niebuhr, Armand Philippot, Chris Swithinbank, Emanuele Stoppa, Erika, Florian Lefebvre, Fred Schott, HiDeoo, Luiz Ferraz, Matt Kane, Matthew Phillips, Reuben Tier, Sarah Rainsberger, and Yan Thomas.

Special thanks to everyone who contributed to Astro 7 with code, docs, reviews, and testing, including:

0x K., 0xRozier, AceCodePt, Adam Chalemian, Adam Matthiesen, Adam McKee, Adam Page, Agus Setiawan, Ahmad Yasser, Alejandro Romano, Alex, Alex Dombroski, Alex Launi, Alexander Flodin, Alexis Aguilar, Aly Cerruti, Amar Reddy, Andreas Deininger, Andrei Alba, Antony Faris, Ariel K, Ash Hitchcock, atsbob, B Sai Thrishul, Barry, Ben Limmer, Bernd Strehl, BitToby, btea, Burra Karthikeya, buschtrisha77, Calvin Liang, Cameron Pak, Cameron Smith, Carlos Lázaro Costa, chaegumi, Chan, Chase McCoy, ChrisLaRocque, Ciaran Moran, Corbin Crutchley, CyberFlame, Daniel Bodky, Daniel Lo Nigro, Daniel Zamyatin, Daniil Sivak, Dario Piotrowicz, dataCenter430, Dawid Gaweł, Desel72, dfedoryshchev, Dom Christie, done, Dor Alagem, Dream, Ed Melly, Ed Melly, Edgar, Ellie, Em Poulter, Eric Grill, Eric Mika, Eryk Baran, Eveeifyeve, fabon, farshad, Felipe Arce, Felix Schneider, Felmon, fkatsuhiro, G Taki@MAX, Giray, Gokhan Kurt, Great Journey, Greg L. Turnquist, Harsh Agarwal, Haz, helio-cf, Henri Fournier, Henri Koskenranta, Henry, Igor Koop, Jack Lukic, Jack Moorhouse, Jack Shelton, jahndan, James Basoo, James Garbutt, James Murty, James Opstad, Jason P. Cochrane, Jett Way, Jimmy, joel hansson, John Mortlock, Johnny Noble, Jon Ege Ronnenberg, Joost de Valk, Jordan Demaison, Josh Soref, JPette1783, Julian Wolf, Julien Cayzac, Junseong Park, Justin Francos, Kai, kato takeshi, Kedar Vartak, Kendell, Kevin Brown, knj, Koji Wakamiya, Konrad Szajna, Kristijan, KTrain, Kumar Gautam, Kyle McLean, Lee Freeman, Leif Marcus, Leonie, Lieke, liruifengv, LongYC, Louis Escher, Luke Deen Taylor, MA2153, Mark Ignacio, Martin DONADIEU, Martin Heidegger, Martin Trapp, Matheus Baroni, Mathieu Mafille, Matthew Justice, Matthias, Matthieu Tremblay, Mavik, Max Malkin, maxim, meyer, Michael Giraldo, Mike Pagé, Milo, Minh Lê, Misrilal, MkDev11, Mochammad Farros Fatchur Roji, moktamd, Naren, Natan Sągol, Nicolò Paternoster, Ntale Swamadu, oab24413gmai, ocavue, Oliver Speir, oliverlynch, Ossaid, Patrick Linnane, Peter Philipp, Phaneendra, Pierre G., pierreeurope, Quetzal Rivera, R A, Raanelom, Rafael Yasuhide Sudo, Rahul Dogra, Rayan Salhab, Rodrigo Santos, Rohan Santhosh Kumar, Roman, Roman Kholiavko, Sam Richard, sanchezmaldonadojesusadrian14-coder, Sanjaiyan Parthipan, sanjibani, Schahin, Sebastian Beltran, Sebastien Barre, Shinya Fujino, Stefan Machhammer, Stel Clementine, Tay, Tee Ming, thelazylama, Timo Behrmann, tmimmanuel, Tobias Breit,

Tom Callahan, Tomasz Cz-Sokołow, travisBREAKS, Tristan Bessoussa, Umut Keltek, Utpal Sen, Vagno, Varun Chawla, Victor Berchet, Vladyslav Shevchenko, Yagiz Nizipli, yy, and 翠

We hope you enjoy Astro 7. If you run into issues or want to share feedback, please join us on Discord, post on GitHub, or reach out on Bluesky, Twitter, and Mastodon.

1. The savings are visible in expressions-dense pages. Static pages wouldn't see much savings. ↩

Starter templates Agencies

Legal

Telemetry

Privacy Policy

Terms of Service
